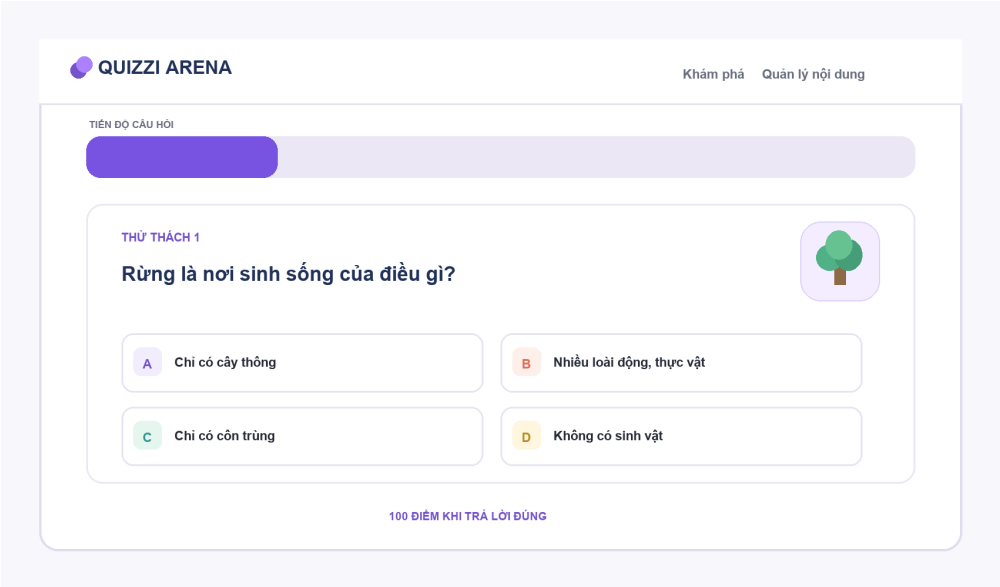
[TÊN TRƯỜNG / CƠ SỞ ĐÀO TẠO]

[KHOA / BỘ MÔN]

BÁO CÁO ĐỒ ÁN

QUIZZI ARENA

HỆ THỐNG TẠO VÀ TỔ CHỨC CÂU HỎI TRẮC NGHIỆM THEO CHỦ ĐỀ



Minh họa bố cục màn chơi Quizzi Arena

Sinh viên thực hiện: [Họ và tên]

Mã sinh viên: [Mã sinh viên]

Lớp: [Tên lớp]

Giảng viên hướng dẫn: [Tên giảng viên]

[ĐỊA ĐIỂM] - NĂM 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn các thầy cô và những người đã hỗ trợ quá trình học tập, tìm hiểu kiến thức phát triển ứng dụng web và hoàn thiện báo cáo này. Những kiến thức về mô hình dữ liệu, lập trình phía máy chủ, giao diện web và triển khai ứng dụng là nền tảng để phân tích Quizzi Arena một cách có hệ thống.

Báo cáo được xây dựng từ mã nguồn của dự án Quizzi Arena và tài liệu hướng dẫn hình thức báo cáo do người thực hiện cung cấp. Các nội dung kỹ thuật được trình bày theo phạm vi mã nguồn và cấu hình hiện có; những phần cần thông tin cá nhân hoặc quy định riêng của cơ sở đào tạo được để ở dạng chỗ trống để người nộp bổ sung.

CAM ĐOAN

Tôi cam đoan đã rà soát nội dung trước khi nộp; các mô tả về Quizzi Arena phản ánh mã nguồn và cấu hình được dùng làm căn cứ. Những tài liệu tham khảo được liệt kê ở cuối báo cáo. Tôi chịu trách nhiệm về các thông tin cá nhân, phần nhận xét và nội dung được bổ sung vào các vị trí để trống.

Người thực hiện: [Họ và tên] Chữ ký: _____

TÓM TẮT

Quizzi Arena là ứng dụng web trắc nghiệm theo chủ đề được xây dựng bằng Django. Hệ thống cho phép quản trị viên tạo chủ đề, tạo nhiều bộ quiz trong mỗi chủ đề và biên soạn câu hỏi có bốn lựa chọn, một đáp án đúng, điểm số và phần giải thích. Người chơi không cần đăng ký tài khoản; họ chọn bộ quiz, nhập biệt danh và trả lời từng câu trong thời gian quy định. Sau khi hoàn thành, ứng dụng hiển thị điểm, số câu đúng, phần trăm chính xác, giải thích đáp án và bảng xếp hạng của bộ quiz.

Phần máy chủ sử dụng các mô hình Topic, Quiz, Question, Choice, Attempt và Answer. Dữ liệu cục bộ được lưu trong SQLite; Django xử lý nghiệp vụ và quản trị nội dung; Gunicorn phục vụ ứng dụng; Nginx nhận HTTPS tại localhost:8444, chuyển tiếp yêu cầu đến ứng dụng và phục vụ tài nguyên tĩnh. Docker Compose tạo mạng nội bộ cùng các volume riêng cho cơ sở dữ liệu, static files và chứng thư localhost.

Báo cáo trình bày yêu cầu, kiến trúc, mô hình dữ liệu, các luồng nghiệp vụ, cách tính điểm, quy trình khởi động bằng Docker, kết quả kiểm tra cấu hình và các giới hạn hiện tại. Qua đó, tài liệu giúp người đọc hiểu cách hệ thống được tổ chức và những bước cần thiết để tiếp tục mở rộng.

Từ khóa: Django, quiz, chủ đề, câu hỏi trắc nghiệm, SQLite, Docker Compose, Nginx.

MỤC LỤC

Lời mở đầu 1

Chương 1. Tổng quan vấn đề nghiên cứu 2

1.1. Bối cảnh và bài toán 2

1.2. Mục tiêu, phạm vi và phương pháp 3

1.3. Tác nhân và yêu cầu chức năng 4

1.4. Yêu cầu phi chức năng và lựa chọn công nghệ 5

Chương 2. Phân tích và thiết kế hệ thống 6

2.1. Kiến trúc tổng thể 6

2.2. Luồng nghiệp vụ 7

2.3. Mô hình dữ liệu 8

2.4. Tuyến URL, phiên chơi và bảo vệ dữ liệu 9

Chương 3. Xây dựng, triển khai và đánh giá 10

3.1. Giao diện và cấu trúc triển khai 10

3.2. Quản lý chủ đề và biên soạn quiz 11

3.3. Tổ chức lượt chơi và tính điểm 12

3.4. Triển khai và vận hành bằng Docker 13

3.5. Kiểm tra, đánh giá và hướng phát triển 14

Kết luận 15

Tài liệu tham khảo 15

DANH MỤC TỪ VIẾT TẮT

Bảng DM.1. Từ viết tắt sử dụng trong báo cáo

Viết tắt	Diễn giải
ORM	Object-Relational Mapping - ánh xạ đối tượng với cơ sở dữ liệu
TLS	Transport Layer Security - giao thức bảo mật đường truyền
HTTPS	HTTP được bảo vệ bằng TLS
CSRF	Cross-Site Request Forgery - giả mạo yêu cầu liên trang
UI	User Interface - giao diện người dùng
DB	Database - cơ sở dữ liệu
CRUD	Create, Read, Update, Delete - tạo, đọc, cập nhật, xóa

DANH MỤC HÌNH VẼ

• Hình 1.1. Tác nhân và phạm vi chức năng (tr. 4)
• Hình 2.1. Kiến trúc triển khai Quizzi Arena (tr. 6)
• Hình 2.2. Luồng nghiệp vụ người chơi và quản trị viên (tr. 7)
• Hình 2.3. Mô hình thực thể và quan hệ (tr. 8)
• Hình 3.1. Bố cục màn chơi (tr. 10)
• Hình 3.2. Quy trình biên soạn nội dung (tr. 11)
• Hình 3.3. Luồng chấm điểm và kết quả (tr. 12)
• Hình 3.4. Khởi động và vận hành local (tr. 13)

DANH MỤC BẢNG BIỂU

• Bảng 1.1. Yêu cầu chức năng (tr. 4)
• Bảng 1.2. Công nghệ và vai trò (tr. 5)
• Bảng 2.1. Thực thể dữ liệu (tr. 8)
• Bảng 2.2. Tuyến URL chính (tr. 9)
• Bảng 3.1. Quy tắc kiểm tra dữ liệu (tr. 11)
• Bảng 3.2. Kết quả kiểm tra xác nhận (tr. 14)
• Bảng 3.3. Giới hạn và hướng mở rộng (tr. 14)

LỜI MỞ ĐẦU

Lý do chọn đề tài

Câu hỏi trắc nghiệm là hình thức phổ biến để ôn tập kiến thức, tổ chức hoạt động lớp học và tự đánh giá mức độ ghi nhớ. Một hệ thống quiz có chủ đề, giới hạn thời gian, phản hồi sau câu trả lời và bảng xếp hạng có thể biến việc luyện tập thành hoạt động tương tác. Dự án Quizzi Arena giải quyết nhu cầu đó bằng một ứng dụng web độc lập, có thể chạy tại máy cá nhân bằng Docker Desktop.

Mục tiêu thực hiện

Mục tiêu của báo cáo là mô tả đúng sản phẩm đang có: thiết kế dữ liệu cho chủ đề, bộ quiz, câu hỏi, lựa chọn và lượt chơi; trình bày các luồng dành cho người chơi và quản trị viên; giải thích cơ chế giới hạn thời gian, lưu đáp án và tính điểm; đồng thời diễn giải cách Docker Compose phối hợp Django, Gunicorn và Nginx.

Phương pháp và phạm vi

Báo cáo được thực hiện bằng cách đọc cấu hình, mô hình, migration, view, URL, template, CSS, JavaScript và các tệp batch của dự án. Sau đó, các thành phần được nhóm theo yêu cầu, dữ liệu, giao diện, triển khai và kiểm tra. Phạm vi đánh giá bao gồm ứng dụng chạy cục bộ, bộ quiz mẫu về rừng và động vật, chức năng tạo nội dung của quản trị viên và lưu dữ liệu bằng SQLite. Báo cáo không xem đây là dịch vụ triển khai công khai trên Internet.

Bố cục gồm ba chương: Chương 1 xác định bối cảnh, mục tiêu và yêu cầu; Chương 2 phân tích kiến trúc, luồng và dữ liệu; Chương 3 mô tả triển khai, vận hành, kiểm tra và hướng mở rộng.

CHƯƠNG 1. TỔNG QUAN VẤN ĐỀ NGHIÊN CỨU

1.1. Bối cảnh và bài toán

Hoạt động ôn tập thường cần ba khả năng: tổ chức câu hỏi theo chủ đề, cho người học biết thời gian còn lại và cung cấp phản hồi sau khi làm xong. Nếu nội dung chỉ được lưu rời rạc hoặc nhập bằng trang quản trị mặc định, người biên soạn khó kiểm soát cấu trúc của một bộ quiz và người chơi khó hình dung tiến độ của lượt làm.

Quizzi Arena xây dựng một thư viện quiz công khai theo chủ đề. Mỗi bộ quiz có tên, mô tả, thời gian cho từng câu và trạng thái công khai. Mỗi câu gồm nội dung, bốn lựa chọn, một đáp án đúng, điểm và phần giải thích tùy chọn. Người chơi nhập biệt danh thay vì tạo tài khoản; kết quả có thể được so sánh trên bảng xếp hạng gắn với bộ quiz.

Vấn đề kỹ thuật trọng tâm là phối hợp dữ liệu có quan hệ với một luồng chơi nhiều bước. Một lượt chơi phải nhận diện đúng quiz, thứ tự câu, câu đã trả lời, thời gian của câu hiện tại và điểm đã cộng. Cùng lúc, khu vực quản trị phải xác thực quyền staff, kiểm tra dữ liệu đầu vào và tránh tạo bộ quiz không có câu hỏi hoặc câu hỏi thiếu lựa chọn.

1.1.1. Định hướng giải quyết

Giải pháp chọn Django để biểu diễn dữ liệu và xử lý yêu cầu phía máy chủ; template Django dựng giao diện; JavaScript hỗ trợ bộ soạn câu hỏi và đồng hồ phía trình duyệt. SQLite đáp ứng lưu trữ cho bản chạy cá nhân. Docker Compose đóng gói các dịch vụ và dữ liệu; Nginx làm điểm truy cập HTTPS cục bộ. Cách tổ chức này giảm số bước cài đặt lặp lại trên máy Windows nhưng vẫn giữ mã nguồn độc lập trong thư mục dự án.

1.2. Mục tiêu, phạm vi và phương pháp

1.2.1. Mục tiêu chức năng

- Cho phép khách duyệt các chủ đề và quiz đã công khai, tìm kiếm quiz theo tên hoặc chủ đề và xem mô tả trước khi chơi.
- Cho phép quản trị viên đăng nhập, tạo chủ đề với tên, mô tả, biểu tượng và màu sắc; tạo nhiều bộ quiz trong một chủ đề.
- Cho phép biên soạn từ 1 đến 50 câu mỗi bộ; mỗi câu có bốn lựa chọn, một đáp án được đánh dấu đúng, số điểm và giải thích tùy chọn.
- Cho phép người chơi nhập biệt danh, trả lời trong thời gian quy định, nhận điểm và xem lại từng câu sau khi hoàn tất.
- Lưu quiz, nội dung câu hỏi, lượt chơi và điểm số qua các lần tắt rồi mở lại Docker Compose.

1.2.2. Phạm vi và giả định

Đối tượng sử dụng gồm khách tham gia quiz và quản trị viên có quyền staff. Quiz mẫu ban đầu gồm mười câu về hệ sinh thái rừng, động vật, quang hợp, phát tán hạt và hành vi bảo vệ thiên nhiên. Quản trị viên có thể tạo nội dung mới nhưng phiên bản hiện tại chưa có màn hình sửa quiz đã lưu, xuất dữ liệu hay quản lý tài khoản người chơi.

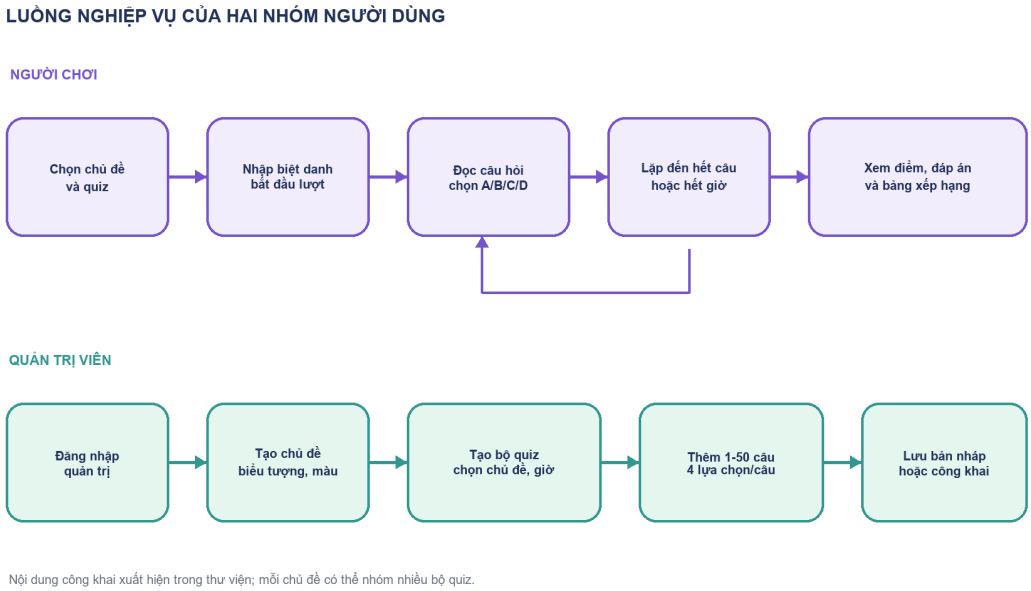
Mỗi câu hiện có đúng bốn lựa chọn và một đáp án đúng. Thời gian được đặt chung cho từng bộ quiz, không phải cấu hình riêng cho từng câu. Điểm được cộng khi trả lời đúng; hệ thống chưa cộng thưởng tốc độ. SQLite và HTTPS tự ký được dùng cho môi trường local, không phải cấu hình vận hành cho lượng truy cập lớn hoặc website công khai.

1.2.3. Phương pháp thực hiện

Quá trình phân tích đi từ cấu trúc bảng dữ liệu đến các view và URL, rồi đối chiếu với mẫu giao diện, Dockerfile, compose.yaml và các batch script. Các nhận định kiểm thử được giới hạn ở những lệnh kiểm tra cấu hình và smoke check đã thực hiện; không suy diễn thành kết quả kiểm thử tải hoặc kiểm thử bảo mật toàn diện.

1.3. Tác nhân và yêu cầu chức năng

Khách không cần đăng nhập để chơi quiz công khai. Quản trị viên đăng nhập qua Django Authentication và phải có cờ staff để mở trang quản lý. Tác nhân quản trị cũng có thể mở Django Admin nâng cao tại /django-admin/; giao diện quản lý thân thiện chính nằm dưới /staff/.



Hình 1.1. Tác nhân và các luồng chức năng chính

Bảng 1.1. Yêu cầu chức năng chính

Mã	Yêu cầu	Kết quả mong đợi
FR-01	Duyệt và lọc nội dung	Hiện quiz công khai; lọc theo chủ đề và từ khóa.
FR-02	Tạo chủ đề	Lưu tên, mô tả, biểu tượng và màu; nhóm được nhiều quiz.
FR-03	Tạo bộ quiz	Gắn vào chủ đề, đặt tên, mô tả, thời gian và trạng thái.
FR-04	Biên soạn câu	Từ 1-50 câu; bốn lựa chọn; chọn đáp án đúng; điểm và giải thích.
FR-05	Tham gia lượt chơi	Nhập biệt danh, nhận câu theo thứ tự, hết giờ thì ghi nhận câu sai.
FR-06	Tổng kết	Hiện tổng điểm, số câu đúng, phần trăm, giải thích và xếp hạng.

Các yêu cầu trên được thể hiện trực tiếp trong view của ứng dụng và biểu mẫu quản trị. Những thao tác tạo hoặc xóa dữ liệu dùng phương thức POST; truy cập khu vực quản lý yêu cầu người dùng đăng nhập và kiểm tra quyền staff.

1.4. Yêu cầu phi chức năng và lựa chọn công nghệ

Ngoài chức năng, phiên bản local cần dễ chạy lại, giữ dữ liệu sau khi dừng container, tách cổng ứng dụng khỏi truy cập trực tiếp và hiển thị tốt trên màn hình nhỏ. Giao diện dùng màu tím, thẻ nội dung, biểu tượng chủ đề, trạng thái thời gian và các hiệu ứng chuyển động nhẹ. Tài nguyên CSS và JavaScript được Nginx phục vụ từ volume tĩnh dùng chung.

Bảng 1.2. Công nghệ và vai trò trong dự án

Thành phần	Vai trò	Căn cứ trong mã nguồn
Django	Mô hình dữ liệu, URL, xác thực, view và template	requirements.txt; quizzi/settings.py
SQLite	Cơ sở dữ liệu local	QUIZZI_DB_PATH=/app/data/db.sqlite3
Gunicorn	WSGI server cho Django	Dockerfile, 2 worker
Nginx	HTTPS, reverse proxy, static files	infra/nginx/default.conf
Docker Compose	Dựng app, nginx, cấp chứng thư và volumes	compose.yaml
JavaScript/CSS	Bộ soạn câu hỏi, đồng hồ và giao diện	quizzes/static/quiz/

1.4.1. Tiêu chí chất lượng

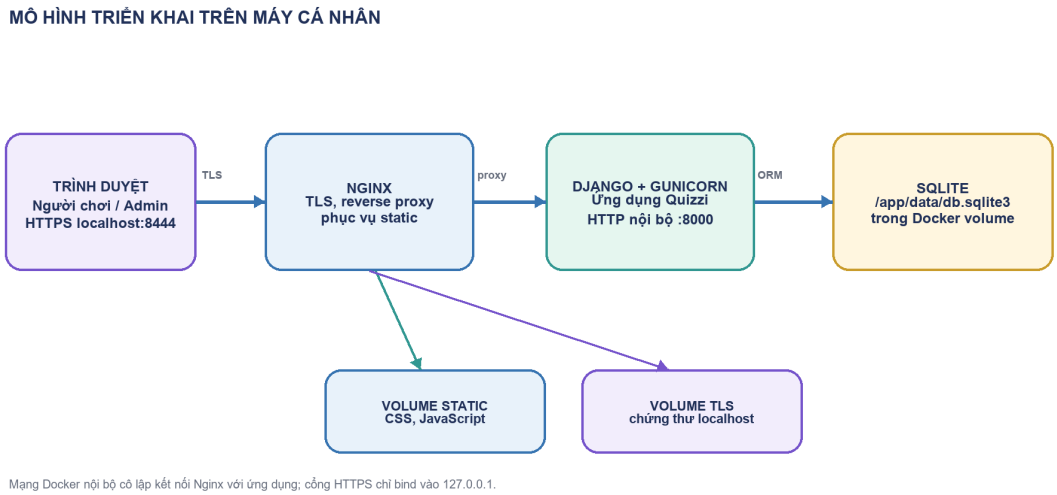
Tính toàn vẹn được hỗ trợ bằng khóa ngoại, migration và ràng buộc mỗi lượt chỉ có một bản ghi trả lời cho một câu hỏi. Tính sẵn sàng ở phạm vi local dựa trên health check của app và restart policy. Tính bảo vệ được hỗ trợ bởi CSRF middleware, đăng nhập staff, HTTPS giữa trình duyệt với Nginx và mạng Docker nội bộ. Khả năng mở rộng vẫn có giới hạn bởi SQLite, cấu hình local và cách quản trị quiz hiện tại.

requirements.txt chấp nhận Django từ 5.2 đến trước 7.0 và Gunicorn 23.x; phiên bản được cố định chưa được khai báo bằng lock file. Vì vậy, lần cài mới có thể nhận phiên bản phụ khác trong dải cho phép. Đây là điểm cần cải thiện khi muốn tái lập build nghiêm ngặt.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Kiến trúc tổng thể

Hệ thống có hai vùng chính: trình duyệt trên máy người dùng và các dịch vụ trong Docker. Trình duyệt kết nối Nginx bằng HTTPS tại cổng 8444. Nginx chuyển yêu cầu động tới dịch vụ app qua HTTP trên mạng nội bộ; app chạy Gunicorn và Django ở cổng 8000, không công bố cổng này ra máy chủ. Các yêu cầu dữ liệu được Django ORM xử lý với SQLite trong volume riêng.

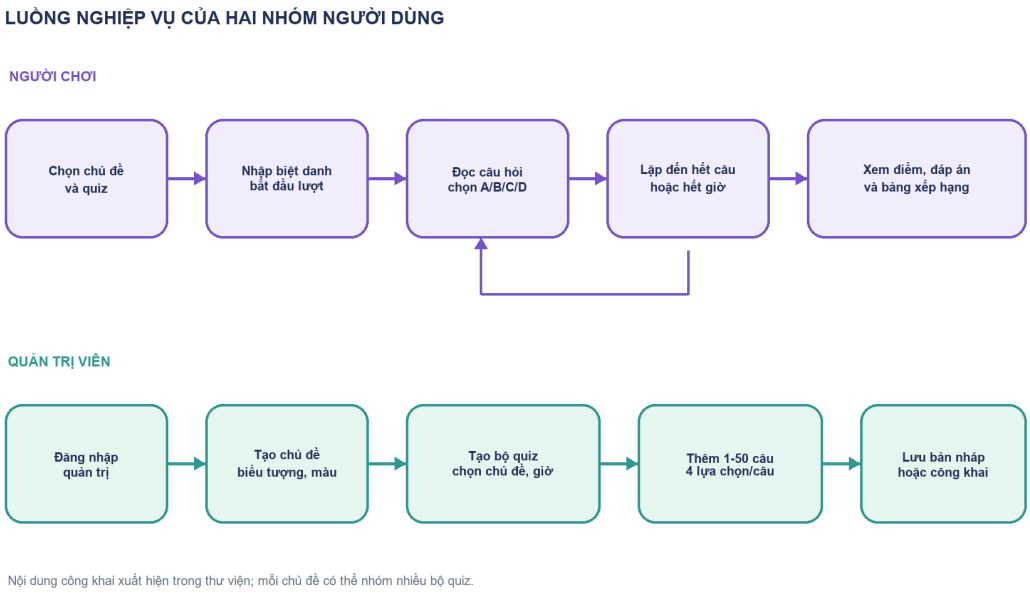


Hình 2.1. Kiến trúc triển khai Quizzi Arena

Certificate-init tạo chứng thư tự ký cho localhost nếu volume chứng thư chưa có tệp hợp lệ. Nginx chỉ bind cổng máy chủ vào 127.0.0.1:8444, phù hợp với mục tiêu chạy local. Volume static được gắn đồng thời vào app và Nginx; volume TLS được Nginx đọc ở chế độ chỉ đọc. Mạng `internal` giới hạn trao đổi giữa các container dịch vụ. Cấu hình này tránh mở trực tiếp Gunicorn, đồng thời tách dữ liệu bền vững khỏi vòng đời container.

Kiến trúc phản ánh một ứng dụng monolith: các chức năng nội dung, lượt chơi và xếp hạng nằm trong cùng dự án Django. Mô hình này đơn giản để cài đặt và bảo trì trong phạm vi đồ án; khi tăng tải, cơ sở dữ liệu và dịch vụ tác vụ có thể được tách ra sau khi có nhu cầu đo đạc thực tế.

2.2. Luồng nghiệp vụ



Hình 2.2. Luồng xử lý dành cho người chơi và quản trị viên

2.2.1. Luồng người chơi

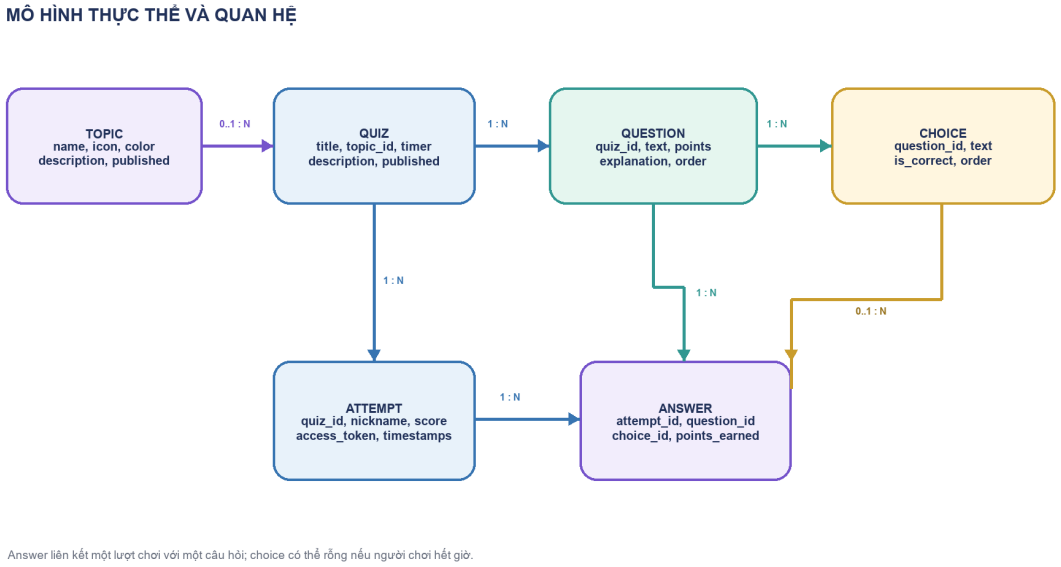
Người chơi mở trang khám phá, chọn chủ đề hoặc tìm quiz, xem mô tả rồi nhập biệt danh. Máy chủ tạo một Attempt và đặt mã truy cập vào session của trình duyệt. Mỗi lần mở trang chơi, view xác định câu kế tiếp từ số câu trả lời đã lưu; template gửi mã câu hỏi cùng lựa chọn được chọn. Nếu hết giờ trước khi gửi lựa chọn, JavaScript gửi biểu mẫu không có đáp án và máy chủ lưu câu đó là sai. Sau câu cuối, Attempt được đánh dấu hoàn tất và trang kết quả truy vấn các đáp án để hiển thị tổng kết.

2.2.2. Luồng quản trị

Quản trị viên tạo chủ đề trước, sau đó chọn chủ đề trong biểu mẫu tạo quiz. Bộ biên soạn dùng JavaScript để thêm hoặc xóa câu hỏi ngay trên trang; dữ liệu được gom thành JSON ẩn và gửi trong một yêu cầu POST. View kiểm tra dữ liệu toàn bộ trước khi mở transaction, sau đó tạo Quiz, Question và Choice. Transaction giúp tránh lưu một bộ quiz chỉ có một phần câu hỏi khi dữ liệu không hợp lệ hoặc ghi cơ sở dữ liệu thất bại.

2.3. Mô hình dữ liệu

Topic nhóm các bộ quiz theo lĩnh vực. Quiz chứa thông tin bộ đề và thời gian chung mỗi câu. Question thuộc một quiz, có thứ tự, điểm và lời giải thích. Choice thuộc một câu hỏi và lưu cờ đánh dấu lựa chọn đúng. Attempt biểu diễn một lần chơi; Answer lưu lựa chọn, kết quả đúng sai và điểm đạt cho mỗi câu trong lượt đó.



Hình 2.3. Quan hệ giữa chủ đề, quiz, câu hỏi và lượt chơi

Bảng 2.1. Thực thể và trách nhiệm dữ liệu

Thực thể	Dữ liệu tiêu biểu	Quan hệ / quy tắc
Topic	name, description, icon, color, is_published	Một chủ đề có nhiều quiz; quiz có thể chưa phân loại.
Quiz	title, subject, seconds_per_question, is_published	Chứa nhiều câu hỏi và nhiều lượt chơi.
Question	text, explanation, points, order	Thuộc một quiz; có nhiều lựa chọn và đáp án.
Choice	text, is_correct, order	Mỗi câu hiện cần bốn lựa chọn.
Attempt	nickname, score, access_token, timestamps	Thuộc một quiz; mã truy cập là UUID.
Answer	is_correct, points_earned, answered_at	Unique(attempt, question); choice có thể rỗng.

Ràng buộc `unique_quiz_answer` ngăn lưu hai câu trả lời cho cùng một câu trong một Attempt. Quan hệ `SET_NULL` giữa Quiz và Topic giữ quiz khi chủ đề bị xóa; thao tác xóa chủ đề trong giao diện quản lý cập nhật các quiz liên quan sang trạng thái chưa phân loại trước khi xóa. Các quan hệ chứa dữ liệu phụ thuộc dùng `CASCADE`, nên xóa quiz cũng xóa câu hỏi, lựa chọn, lượt chơi và đáp án của quiz đó.

2.4. Tuyến URL, phiên chơi và bảo vệ dữ liệu

Bảng 2.2. Các tuyến URL tiêu biểu

Đường dẫn	Phương thức / quyền	Chức năng
/	GET, công khai	Danh sách chủ đề và quiz.
/quiz/<id>/	GET	Chi tiết, form biệt danh và bảng xếp hạng.
/quiz/<id>/start/	POST	Tạo Attempt mới.
/quiz/<id>/play/	GET/POST	Hiện câu và ghi lựa chọn.
/result/<token>/	GET, session khớp	Kết quả lượt chơi.
/staff/topics/create/	POST, staff	Tạo chủ đề.
/staff/quizzes/create/	POST, staff	Tạo quiz và câu hỏi.
/django-admin/	GET/POST, staff	Giao diện Django Admin nâng cao.

2.4.1. Quản lý tiến độ và quyền

Token UUID của Attempt không được sửa sau khi tạo. Token được lưu trong session theo mã quiz; trang kết quả so sánh token trong session với lượt chơi tương ứng trước khi trả nội dung. Cách này hạn chế truy cập kết quả chỉ bằng một URL đoán được. Biệt danh được rút gọn tối đa 40 ký tự. Câu hỏi chỉ được ghi nhận một lần nhờ ràng buộc duy nhất ở cơ sở dữ liệu.

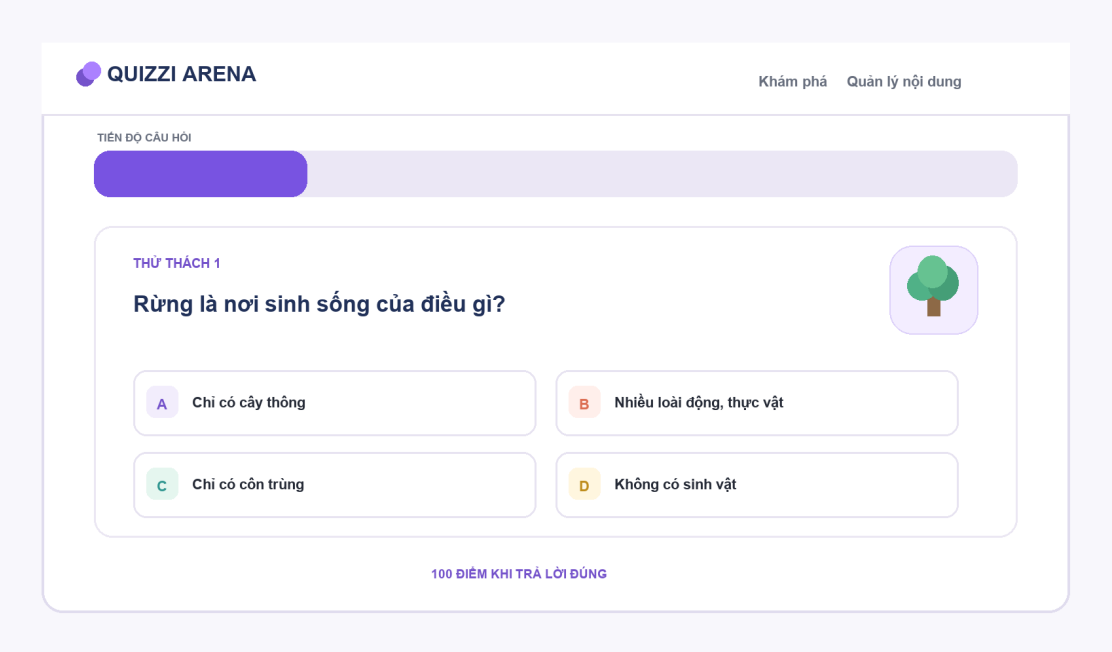
Các view biên soạn yêu cầu đăng nhập và kiểm tra `is_staff`. Các form dùng CSRF token; các tuyến tạo/xóa và gửi câu hỏi được giới hạn bằng POST. Django middleware có CSRF, session và authentication. `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS` và proxy HTTPS được cấu hình qua biến môi trường. Trong môi trường local, chứng thư tự ký khiến trình duyệt báo chưa xác thực CA; đây là đặc tính của cấu hình thử nghiệm, không phải chứng thư thương mại.

Một hạn chế ở lớp trình duyệt là đồng hồ JavaScript có thể bị can thiệp; máy chủ vẫn kiểm tra thời gian dựa trên `question_started_at` trước khi nhận lựa chọn. Với nhiều người dùng đồng thời, SQLite và dữ liệu lượt chơi cần được đánh giá tải, bổ sung giới hạn yêu cầu và sao lưu trước khi đưa ra Internet.

CHƯƠNG 3. XÂY DỰNG, TRIỂN KHAI VÀ ĐÁNH GIÁ

3.1. Giao diện và cấu trúc triển khai

Giao diện được chia thành thư viện quiz, trang chi tiết, trang chơi, trang kết quả và khu vực quản lý nội dung. Template dùng chung đặt trong `quizzes/templates/quiz/`; CSS và JavaScript được lưu trong `quizzes/static/quiz/`. Tệp base.html dựng thanh điều hướng và vùng nội dung; list.html tạo thẻ quiz và bộ lọc chủ đề; detail.html cung cấp phần giới thiệu và nhập biệt danh; play.html hiển thị câu, đáp án, đồng hồ; result.html trình bày tổng kết và giải thích.



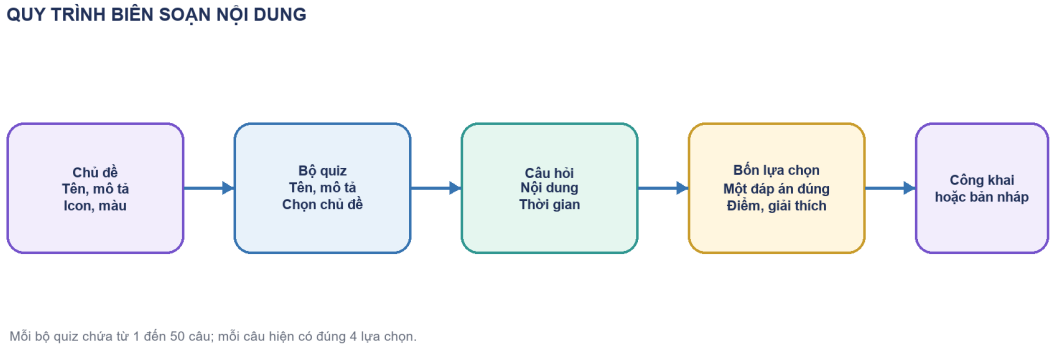
Hình 3.1. Minh họa bố cục màn chơi Quizzi Arena

Màn chơi phân cấp thông tin theo thứ tự: tên quiz và thoát lượt, thanh tiến độ, số câu và thời gian, nội dung câu hỏi, bốn lựa chọn và điểm thưởng. Các thẻ lựa chọn có màu định danh, trạng thái hover và hiệu ứng xuất hiện. Trang quản lý nội dung dùng cùng bộ màu nhưng tổ chức thành các khu vực chủ đề, bộ quiz và bộ soạn câu hỏi. CSS có breakpoint cho màn hình nhỏ và tôn trọng `prefers-reduced-motion` để hạn chế chuyển động theo thiết lập người dùng.

Các minh họa trong báo cáo là sơ đồ giải thích bố cục và luồng, không phải ảnh chụp kết quả kiểm thử trên thiết bị. Mã giao diện thực tế có thể thay đổi sau khi người quản trị cập nhật nội dung hoặc cấu hình triển khai.

3.2. Quản lý chủ đề và biên soạn quiz

Quản trị viên sử dụng form tạo chủ đề để khai báo tên, mô tả, biểu tượng và màu. Tên được chuẩn hóa khoảng trắng và giới hạn 80 ký tự; backend từ chối chủ đề trùng không phân biệt hoa thường. Sau khi lưu, trang quản lý hiển thị số bộ quiz và số câu được nhóm trong mỗi chủ đề.



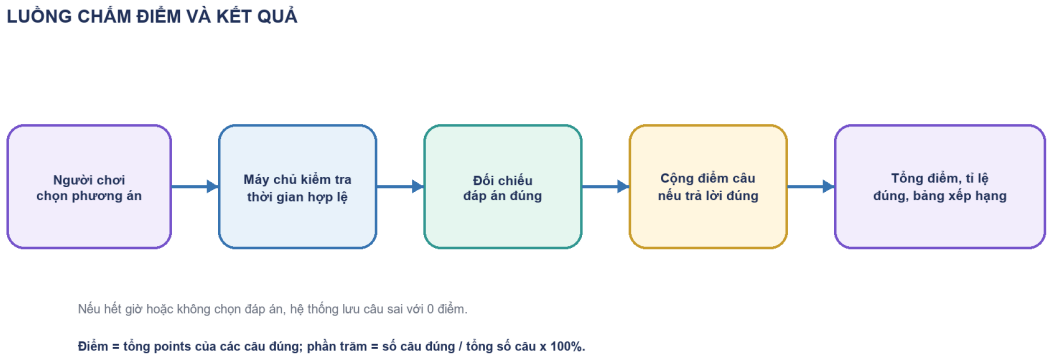
Hình 3.2. Quy trình từ chủ đề đến bộ quiz có nhiều câu hỏi

Bảng 3.1. Kiểm tra dữ liệu của bộ biên soạn

Dữ liệu	Quy tắc phía máy chủ	Phản hồi
Chủ đề và tên quiz	Chủ đề phải tồn tại; tên không được rỗng.	Yêu cầu chọn chủ đề và nhập tên.
Thời gian	Từ 5 đến 180 giây mỗi câu.	Từ chối cấu hình ngoài miền.
Số câu	Danh sách JSON có 1-50 phần tử.	Yêu cầu thêm ít nhất một câu.
Mỗi câu	Nội dung, bốn đáp án không rỗng, chỉ số đúng 0-3.	Không lưu quiz nếu thiếu dữ liệu.
Điểm và lời giải	Điểm 0-1.000; giải thích tối đa 2.000 ký tự.	Giới hạn trước khi lưu.

`editor.js` quản lý danh sách câu hỏi trong trình duyệt, tạo thẻ nhập nội dung, bốn lựa chọn, radio chọn đáp án đúng, điểm và phần giải thích. Khi người dùng gửi form, danh sách được chuyển thành JSON. Backend không tin dữ liệu từ JavaScript mà kiểm tra lại kiểu, số lượng, độ dài và miền giá trị. Quiz và các câu được tạo trong một transaction để giữ tính nhất quán.

3.3. Tổ chức lượt chơi và tính điểm



Hình 3.3. Luồng ghi nhận câu trả lời và tổng kết

Khi bắt đầu, hệ thống tạo Attempt với biệt danh, mã UUID, điểm bằng không và thời điểm câu đầu tiên. Vị trí câu hỏi được suy ra từ số Answer đã lưu. Lựa chọn nhận từ form chỉ được chấp nhận nếu mã câu hỏi khớp với câu đang hiển thị, lựa chọn thuộc chính câu đó và thời gian chưa vượt quá giới hạn của quiz.

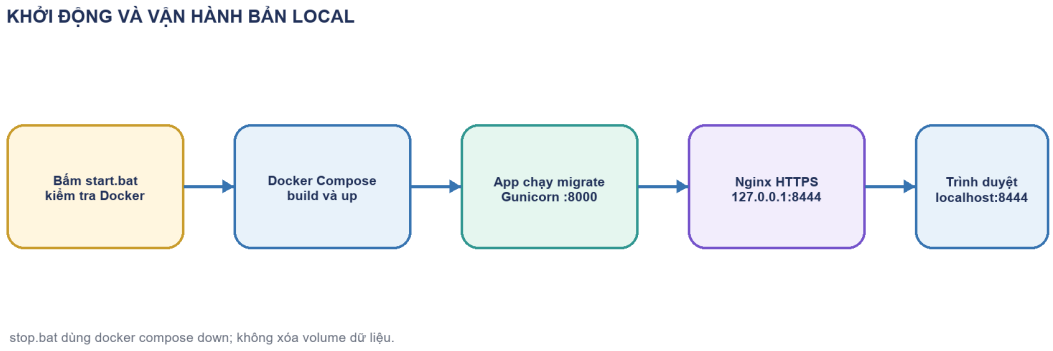
3.3.1. Công thức điểm

Với câu hỏi i có số điểm p_i, điểm nhận ở câu đó bằng p_i nếu câu trả lời đúng, bằng 0 nếu sai hoặc hết giờ. Tổng điểm của lượt là tổng điểm nhận ở tất cả câu. Tỉ lệ chính xác được tính theo số câu đúng chia tổng số câu, nhân 100%. Vì điểm từng câu có thể khác nhau, điểm đạt và phần trăm chính xác là hai chỉ số riêng.

Ví dụ: bộ quiz có 10 câu, mỗi câu 100 điểm. Nếu người chơi đúng 7 câu, điểm là 700/1.000 và độ chính xác là 70%. Nếu các câu có điểm lần lượt khác nhau, hệ thống vẫn cộng theo điểm cấu hình; phần trăm chính xác không thay đổi trọng số theo điểm. Bảng xếp hạng sắp xếp theo điểm giảm dần, sau đó theo thời điểm hoàn tất.

Khi hết thời gian, JavaScript gửi form không có lựa chọn; server ghi câu trả lời sai và đưa tiến độ sang câu kế tiếp. Cách triển khai này đơn giản nhưng phụ thuộc vào lần tải trang và đồng hồ trên máy khách; cải tiến có thể đưa đồng hồ sang cập nhật động phía máy chủ hoặc bổ sung sự kiện reconnect cho lượt chơi dài.

3.4. Triển khai và vận hành bằng Docker



Hình 3.4. Quy trình khởi động Quizzi Arena trên Windows

Dockerfile dùng Python 3.12 slim, cài requirements, sao chép mã nguồn, thu thập static files và chạy Gunicorn với hai worker. Khi container app bắt đầu, lệnh khởi động chạy `python manage.py migrate --noinput` trước Gunicorn. Health check gọi `/healthz/`. Nginx dùng cấu hình TLS, chuyển tiếp header host và scheme, đồng thời đọc static files từ volume dùng chung.

3.4.1. Cách chạy cho người dùng

• Mở Docker Desktop và chờ Docker Engine sẵn sàng.

• Mở `D:\Quizzi-Arena` trong File Explorer hoặc VS Code.

• Bấm đúp `start.bat`; script tạo `.env` từ `.env.example` nếu cần và chạy `docker compose up -d --build`.

• Mở https://localhost:8444. Chứng thư local tự ký có thể làm Edge hiển thị cảnh báo chưa xác thực.

• Lần đầu tạo admin: khi app đã chạy, dùng `setup-admin.bat`; đăng nhập tại `/staff/login/`.

• Tắt bằng `stop.bat`; lệnh `docker compose down` giữ dữ liệu trong volume.

Ba volume có vai trò khác nhau: `quizzi-data` chứa SQLite và trạng thái quiz; `quizzi-static` chứa CSS/JavaScript được Nginx phục vụ; `tls-certs` giữ chứng thư. Không dùng `docker compose down -v` nếu cần giữ dữ liệu. File `.env` chứa khóa bí mật; cần thay khóa mặc định trước khi sao chép hoặc chia sẻ dự án. Hướng dẫn chi tiết hơn nằm trong README.md.

3.5. Kiểm tra, đánh giá và hướng phát triển

Bảng 3.2. Kết quả kiểm tra xác nhận đã thực hiện

Hạng mục	Kết quả quan sát	Kết luận
Django system check	Không phát hiện lỗi cấu hình.	Đạt kiểm tra cấu hình.
Migration check	Không còn thay đổi model chưa có migration.	Lược đồ đồng bộ với model.
Compose config	docker compose config --quiet thành công.	Cấu hình YAML hợp lệ.
Container	App healthy; Nginx đang chạy.	Dịch vụ sẵn sàng local.
Smoke check URL	Trang chủ khách/admin 200; /admin/ chuyển 302 đến dashboard; dashboard 200.	Luồng truy cập chính hoạt động.

Các kết quả trên là kiểm tra cấu hình và smoke check các URL, không phải bộ kiểm thử tự động bao phủ toàn bộ nghiệp vụ, kiểm thử tải hay đánh giá bảo mật độc lập. Lỗi tên reverse URL trong thanh điều hướng staff từng làm trang chủ trả 500 khi admin đăng nhập; sau khi sửa tên route toàn cục, trang chủ và dashboard được xác nhận trả mã trạng thái đúng. Tài liệu cần được kiểm tra lại nếu route hoặc namespace thay đổi.

Bảng 3.3. Giới hạn hiện tại và hướng mở rộng

Giới hạn	Tác động	Hướng tiếp tục
Chưa sửa quiz đã lưu	Sai nội dung phải xóa và tạo lại.	Thêm trang chỉnh sửa quiz/câu hỏi.
Cố định bốn lựa chọn	Không hỗ trợ câu nhiều đáp án hoặc tự luận.	Mở rộng kiểu câu và đáp án.
SQLite và một máy	Chưa phù hợp lưu lượng đồng thời lớn.	Đo tải rồi chuyển PostgreSQL nếu cần.
Chứng thư tự ký	Trình duyệt không tin cậy HTTPS local.	Dùng domain và CA hợp lệ khi public.
Chưa có unit test	Lỗi nghiệp vụ có thể lọt qua smoke check.	Viết test model, view, quyền và scoring.
Chưa có sao lưu tự động	Có thể mất volume nếu người dùng xóa nhầm.	Thêm lệnh backup và hướng dẫn restore.

KẾT LUẬN

Quizzi Arena hiện thực một hệ thống trắc nghiệm web có cấu trúc chủ đề, bộ quiz, câu hỏi và lượt chơi rõ ràng. Quản trị viên có thể tạo nội dung nhiều câu theo chủ đề; người chơi tham gia bằng biệt danh, trả lời có giới hạn thời gian và xem điểm cùng phần giải thích. Mô hình Django, cơ sở dữ liệu SQLite và triển khai Docker Compose đáp ứng mục tiêu sử dụng độc lập trên máy cá nhân; Nginx tạo điểm truy cập HTTPS local và phục vụ tệp tĩnh.

Qua việc phân tích mã nguồn, báo cáo đã làm rõ quan hệ dữ liệu, cách session gắn lượt chơi, quy tắc xác nhận đáp án, công thức điểm và vai trò của từng container. Những kiểm tra đã thực hiện cho thấy cấu hình Django và Compose hợp lệ, dịch vụ app healthy và các luồng URL chính trả trạng thái phù hợp. Tuy nhiên, kiểm tra hiện tại chưa thay thế cho kiểm thử tự động, kiểm thử tải và rà soát bảo mật trước khi đưa ứng dụng ra Internet.

Các hướng phát triển ưu tiên là bổ sung chỉnh sửa quiz, tăng tính linh hoạt của loại câu hỏi, viết kiểm thử nghiệp vụ, cải thiện sao lưu dữ liệu và chuẩn hóa biến môi trường. Nếu triển khai công khai, cần dùng cơ sở dữ liệu phục vụ đồng thời tốt hơn, khóa bí mật riêng, HTTPS với chứng thư hợp lệ, giám sát và quy trình phục hồi dữ liệu.

TÀI LIỆU THAM KHẢO

[1] Mã nguồn Quizzi Arena: README.md, requirements.txt, quizzi/, quizzes/, Dockerfile, compose.yaml và infra/nginx/default.conf. Dự án local, rà soát ngày 25/10/2026.

[2] Django Software Foundation, “Migrations”, Django documentation. https://docs.djangoproject.com/en/dev/topics/migrations/ (truy cập 25/10/2026).

[3] Docker, “Docker Compose”. https://docs.docker.com/compose/ (truy cập 25/10/2026).

[4] Docker, “Define and manage volumes in Docker Compose”. https://docs.docker.com/reference/compose-file/volumes/ (truy cập 25/10/2026).

[5] Nginx, “Configuring HTTPS servers”. https://nginx.org/en/docs/http/configuring_https_servers.html (truy cập 25/10/2026).

[6] Tài liệu hướng dẫn định dạng báo cáo và kỹ thuật Word, PDF do người thực hiện cung cấp, các trang in 142-146.